NOTICE TECHNIQUE :
Diagramme de Voronoï - SAE S6

Projet réalisé par :
- Adrian GALINDO
- Bharani RATTINASSABABADY
- Mouhammed Diop
- Anouar ROUIBI
- Maxime LOINTIER
- Huy PHAM

SOMMAIRE

1. Introduction
2. Méthode de construction du diagramme de Voronoï
3. Choix techniques
4. Architecture logicielle
5. Bilan : limites et perspectives

Introduction

Dans le cadre de notre SAE de semestre 6 en BUT Informatique, nous avons relevé le défi de concevoir une application capable de générer et de visualiser un diagramme de Voronoï. L'idée était de partir d'un simple semis de points dans un plan pour aboutir à une partition géométrique précise.

Bien qu'il puisse paraître abstrait, le diagramme de Voronoï est un outil concret utilisé massivement en géographie pour le découpage de zones d'influence, en robotique pour l'évitement d'obstacles, ou encore en biologie. Notre objectif ne se limitait pas au simple calcul mathématique : il nous fallait construire un outil dynamique, doté d'une interface fluide pour que l'utilisateur puisse manipuler ses données (saisie, import, export) sans friction. Cette notice détaille notre cheminement, de la théorie aux choix techniques, sans oublier les limites rencontrées.

Méthode de construction du diagramme de Voronoï

Fondements théoriques

Pour bien comprendre l'outil, il faut revenir à la définition d'un ensemble fini de points $P = \{p_1, p_2, ..., p_n\}$. Une cellule de Voronoï liée à un point p_i regroupe tous les points du plan qui sont plus proches de ce p_i que de n'importe quel autre point p_i de l'ensemble.Mathématiquement. Le résultat final est une partition du plan en régions convexes, créant une structure duale de la triangulation de Delaunay.

Le choix de la triangulation de Delaunay

Plutôt que de calculer les cellules de manière isolée, nous nous sommes appuyés sur la triangulation de Delaunay. Sa propriété fondamentale est qu'aucun point ne doit se trouver à l'intérieur du cercle circonscrit d'un triangle de la triangulation. Cela évite d'avoir des triangles trop fins ou "écrasés", ce qui stabilise les calculs numériques.

Pour l'implémentation, nous avons intégré la bibliothèque d3-delaunay. Ce choix nous a permis de garantir une fiabilité mathématique immédiate sur la

génération des cellules, nous laissant ainsi plus de temps pour nous concentrer sur l'architecture logicielle et l'expérience utilisateur.

Le processus de calcul et normalisation

Dans l'application, le flux est automatique : dès qu'un point est ajouté ou modifié, le système récupère la collection, normalise les coordonnées pour qu'elles rentrent parfaitement dans le canvas HTML, puis redessine l'ensemble.

La phase de normalisation est d'ailleurs cruciale. En calculant les extrema (X et Y) et en ajoutant un léger padding, on s'assure que le diagramme reste lisible et ne vient pas "coller" aux bords de la fenêtre, peu importe la distribution des points en entrée.

Choix techniques

Nous avons rapidement écarté les solutions logicielles lourdes au profit du Web. L'accessibilité est immédiate (pas d'installation) et la compatibilité est totale, que l'on soit sous Windows ou macOS. L'utilisation de l'API Canvas nous permet un rendu graphique réactif, indispensable pour une application qui se veut dynamique.

Côté technologies, nous sommes restés sur un stack moderne : JavaScript (ES Modules) pour la modularité, HTML5/CSS3 pour l'interface, et Jest pour sécuriser notre code via des tests unitaires.

Architecture logicielle

Une organisation modulaire et découplée

Plutôt que de construire un bloc monolithique, nous avons fait le choix d'une architecture modulaire où chaque fichier JavaScript possède une responsabilité unique. Cette approche nous a permis de travailler plus sereinement sur les différentes briques du projet :

- Objets : On a décidé de créer des objets *Point* et *PointCollection* pour le stockage des points.

- Gestion des données : Le module lecture.js fait le pont avec l'utilisateur. Pour garantir une structure de données stable, parsing.js gère la collection centrale des points et le stock dans les objets.

- Logique et Rendu : Le cœur du calcul est encapsulé dans voronoi.js, ce qui isole la bibliothèque externe du reste du code. Le rendu.js pilote l'affichage (normalisation et dessin sur le canvas) et export.js s'occupe de la sortie vers le format PNG.

Cette séparation des responsabilités (SOC) n'est pas qu'une question de propreté : elle nous a permis de faire évoluer le moteur de rendu sans jamais risquer d'altérer la logique de récupération des points.

Flux de fonctionnement interne

Le cycle de vie d'une interaction dans l'application suit un parcours linéaire mais robuste. Tout commence par la saisie ou l'importation. Les données passent immédiatement au crible d'expressions régulières pour valider leur format avant d'intégrer la collection centralisée.

Une fois la collection mise à jour, un "événement" de recalcul est déclenché : le diagramme est généré à nouveau, les coordonnées sont projetées sur le canvas, et le rendu visuel se rafraîchit. L'utilisateur garde alors la main pour exporter son travail si le résultat lui convient. Ce flux tendu limite les effets de bord et facilite grandement la maintenance du code.

Stratégie de test avec Jest

La fiabilité d'une application de géométrie repose sur la précision de ses entrées. Nous avons donc utilisé le framework Jest pour mettre en place une suite de tests unitaires rigoureuse.

L'un de nos choix stratégiques a été d'isoler complètement le parsing du rendu graphique. Cela nous a permis de tester des scénarios complexes (coordonnées mal formées, fichiers corrompus, gestion des erreurs de saisie) sans avoir besoin de simuler un environnement de navigateur. En testant le comportement de la collection de points de manière indépendante, nous nous sommes assurés que la "source de vérité" de l'application restait intègre en toute circonstance. Nous avons aussi testé chaque Objet (*Point* et *PointCollection*) pour nous assurer l'intégralité des nos points.

Bilan : limites et perspectives

Obstacles et limites actuelles

Comme tout projet de développement, nous avons dû faire face à certains compromis techniques. À l'heure actuelle, l'application recalcule l'intégralité du diagramme à chaque petite modification. Si cela est imperceptible avec quelques dizaines de points, les performances commencent à décliner dès que l'on manipule des jeux de données très denses. De plus, l'expérience utilisateur reste perfectible : il manque encore la possibilité de supprimer ou de déplacer un point directement à la souris sur le canvas.

Pistes d'évolution

Pour transformer ce prototype en un outil de production complet, plusieurs axes d'amélioration se dégagent. L'ajout d'un zoom interactif et d'un mode de déplacement dans le plan simplifierait l'exploration des grands diagrammes. Techniquement, l'implémentation d'une mise à jour "incrémentale" permettrait de ne recalculer que les zones modifiées, boostant ainsi les performances. Enfin, une dimension esthétique et pédagogique pourrait être une animation simulant pas à pas la construction de la triangulation.